Flight Simulation with Motorized Gauges and Hardware-Software Integration

Real-Time Flight Instrumentation Cluster System

Author
Kenyi Kubari

BSc. Mechanical Engineering (Minor Computer Science)
Academic year: 2025-2026

Contents

1 Introduction . . . 6
  1.1 Project Proposal . . . 6
2 Background . . . 7
  2.1 Aircraft Flight Instruments . . . 7
  2.2 The Six Primary Flight Instruments . . . 7
3 Methods . . . 9
  3.1 Control System Architecture . . . 9
    3.1.1 Open-Loop vs. Closed-Loop Control . . . 9
    3.1.2 Servo Motors vs. DC Motors: Design Trade-offs . . . 10
  3.2 Motor Control Methodologies . . . 10
    3.2.1 Pulse Width Modulation . . . 10
    3.2.2 Speed Ramping and Smooth Acceleration . . . 11
  3.3 Communication Protocols . . . 11
4 Design . . . 12
  4.1 Hardware Architecture . . . 12
  4.2 Software Architecture . . . 13
  4.3 Software Design Considerations . . . 15
  4.4 Testing and Validation . . . 16
5 Challenges . . . 18
  5.1 Challenge 1: Lack of Position Feedback . . . 18
  5.2 Challenge 2: Crowded Circuit . . . 19
  5.3 Challenge 3: Logging Data for Real-Time Dashboard . . . 19
6 Results and Discussion . . . 20
7 Conclusion . . . 22
  7.1 Lessons Learned . . . 22
  7.2 Future Improvements . . . 22
  7.3 Final Thoughts . . . 23
A Media and Resources . . . xxv
B Documentation and Schematics . . . xxvi
Bibliography . . . xxxi

Table of figures

Figure 1 Hardware Setup: Two Arduino Unos in Parallel Configuration 13
Figure 2 Software Architecture: Data Flow and Component Interaction 14
Figure 3 Flight emulator controller: (a) front view and (b) top view. 21
Figure 4 Arduino RedBoard V2.2 schematic xxvii
Figure 5 TB6612FNG Motor Driver Spec Sheet [1]. xxviii
Figure 6 Flight Emulator Circuit Schematic Page 1 xxix
Figure 7 Flight Emulator Circuit Schematic Page 2 xxx

Table of tables

Table 1 The six primary flight instruments and their functions 8
Table 2 Bill of Materials (BOM): Robot Components and Specifications 12
Table 3 Decision matrix comparing Python, C/C++, and Rust for Arduinos 16

Chapter 1

Introduction

1.1 Project Proposal

I am studying mechanical engineering with an interest in aerospace. For my final project, I wanted to choose a project that combined my interests in aircraft and embedded systems. So I have chosen to build a functional flight emulator that brings together physical analog gauges, a digital display, and a computer dashboard to recreate what it's like to monitor aircraft instruments in real-time. The idea is to take the complexity of an actual cockpit and make it tangible and interactive.

My proposal is to design and build a flight simulation system using Arduino-based microcontrollers, featuring motorized analog gauges that respond to real-time flight parameters. The system will be integrated with a digital display and a software dashboard for visualization, control, and data logging, creating a cohesive hardware-software platform that emulates aircraft instrumentation.

Chapter 2

Background

2.1 Aircraft Flight Instruments

Modern aircraft rely on a standardized suite of flight instruments that allow pilots to safely control and navigate the aircraft, especially under Instrument Flight Rules (IFR). When outside visual references are limited or unavailable, these instruments provide the information needed to maintain situational awareness by indicating the aircraft's position, motion, and orientation. Refined over decades, they form the foundation of safe and effective instrument flying [2].

2.2 The Six Primary Flight Instruments

Rather than attempting to replicate all six instruments in full detail, this project focuses on four core flight parameters: yaw, roll, pitch, and heading [3], [4]. These variables capture the essential aspects of aircraft orientation and directional control while keeping the project scope manageable. A digital display is also included to show the corresponding numerical values, providing clearer feedback and improving usability. This approach balances realism with practical design constraints while still demonstrating the key principles of flight instrumentation.

Table 1: The six primary flight instruments and their functions

Instrument	Purpose
Altimeter	Displays altitude above mean sea level for terrain clearance and navigation
Airspeed Indicator	Indicates aircraft speed relative to the surrounding air to help prevent stalls and structural overstress
Attitude Indicator	Shows pitch and bank angle relative to the horizon for spatial orientation
Heading Indicator	Displays aircraft heading and assists with directional control
Turn Coordinator	Indicates rate of turn and whether the aircraft is in coordinated flight
Vertical Speed Indicator	Shows the rate of climb or descent

Chapter 3

Methods

This section outlines a high-level overview of the methods used in this project, including the hardware and software design choices and control system architecture.

3.1 Control System Architecture

This project implements a control architecture that considers both open-loop and closed-loop control. Understanding both control strategies is important for evaluating the design decisions and trade-offs made throughout the system.

3.1.1 Open-Loop vs. Closed-Loop Control

Control systems are typically classified based on whether feedback is used to adjust system behavior. In this project, control at the microcontroller level is primarily open-loop, with calibration handled at the system level.

Open-loop control: Control actions are applied without measuring the resulting output. For example, when the computer sends a PWM signal to drive a motor to a target position, the Arduino applies the signal without verifying the actual position achieved.

Closed-loop control: The system continuously measures output, compares it to a desired setpoint, and adjusts the control input accordingly. A common example is a thermostat, which regulates temperature by adjusting heating based on feedback. Closed-loop control improves accuracy and robustness but requires additional sensors and processing.

3.1.2 Servo Motors vs. DC Motors: Design Trade-offs

Motor selection plays a significant role in determining the overall control strategy. This project considers both DC gear motors and servo motors.

DC gear motors: DC gear motors are simple electromechanical devices that rotate when voltage is applied. They require an external driver to control direction and speed, with speed regulated through Pulse Width Modulation (PWM). The motors used in this project, DAGU Robot DG01D motors with a 48:1 gearbox they provide sufficient torque for driving gauge needles. However, they do not provide position feedback.

Servo motors: Servo motors integrate a DC motor, gearbox, position sensor, and control electronics into a single unit. Position is controlled using a PWM signal with pulse widths corresponding to specific angles. Internal feedback allows the servo to maintain its commanded position, making it a closed-loop device.

3.2 Motor Control Methodologies

3.2.1 Pulse Width Modulation

Pulse Width Modulation (PWM) is used to control motor speed by varying the effective voltage supplied to the motor. A PWM signal alternates between 0 V and 5 V at a fixed frequency. On the Arduino Uno, PWM outputs operate at approximately 490 Hz with 8-bit resolution from 0 to 255.

The duty cycle, defined as the ratio of the signal's on-time to its total period, determines the effective voltage. For example, a 50% duty cycle corresponds to approximately half the supply voltage.

For DC motors, the duty cycle directly controls rotational speed. In contrast, servo motors interpret pulse width rather than duty cycle, requiring a different signal structure generated through a dedicated library or custom timing.

3.2.2 Speed Ramping and Smooth Acceleration

To improve the visual and mechanical response of the system, motor speed is not changed instantaneously. Instead, speed is increased gradually using a linear ramp:

$$\text{current_speed} = \text{current_speed} + \text{acceleration_increment} \tag{1}$$

This approach reduces abrupt motion and produces smoother transitions that better reflect real mechanical systems. Acceleration rates are tuned experimentally to account for differences in load, friction, and supply conditions.

3.3 Communication Protocols

Communication between the computer and the microcontrollers is implemented using USB serial at 9600 baud. This method is simple, reliable, and sufficient for the relatively low data rates required by the system.

Chapter 4

Design

4.1 Hardware Architecture

The hardware for this project consists of components from two SparkFun Inventor's Kits. The specific components used are outlined in the Bill of Materials in Table 2.

Table 2: Bill of Materials (BOM): Robot Components and Specifications

QTY	Component	Specification
2	Microcontroller	Sparkfun RedBoard
4	DC Motor	DAGU Robot DG01D 48:1 GearBox
2	Motor Driver	Toshiba TB6612FNG
2	LCD Display	16x2 I2C
2	Potentiometer	–
4	Wheels	65mm Diameter

Two Arduinos were used because the project required a large number of components and sensors, exceeding the pin availability of a single board. By using two Arduinos in a parallel (stereo) configuration, responsibilities for displaying flight data were distributed between the boards with minimal overhead and coding complexity.

In this architecture, the computer program running on the PC processes and sends the required flight data to each Arduino. This allows each microcontroller to focus solely on displaying its assigned flight parameters, simplifying the embedded logic.

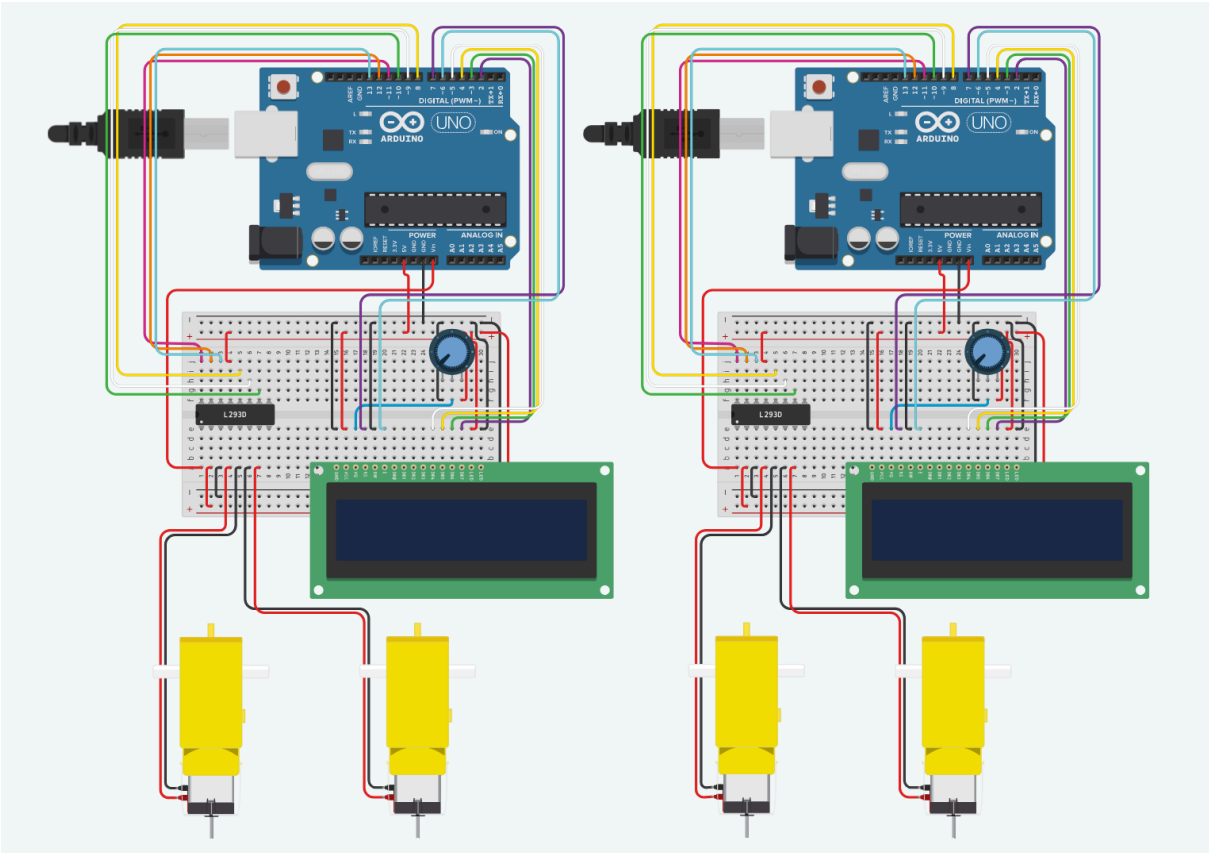


Figure 1: Hardware Setup: Two Arduino Unos in Parallel Configuration

Figure 1 shows the wiring diagram for the two Arduinos. The left Arduino, `controller_1`, is responsible for displaying heading and yaw on the analog gauges, as well as flight speed on the digital display. The right Arduino, `controller_2`, handles roll and pitch on the analog gauges and altitude on the display.

Both Arduinos receive data from the computer via USB serial communication. A more detailed wiring diagram is included in the Appendix (Figure 6 and Figure 7).

4.2 Software Architecture

The software architecture consists of two main layers: the Computer Layer, which handles data processing and dashboard visualization, and the Microcontroller Layer,

which manages the real-time display of flight parameters on the gauges and digital displays.

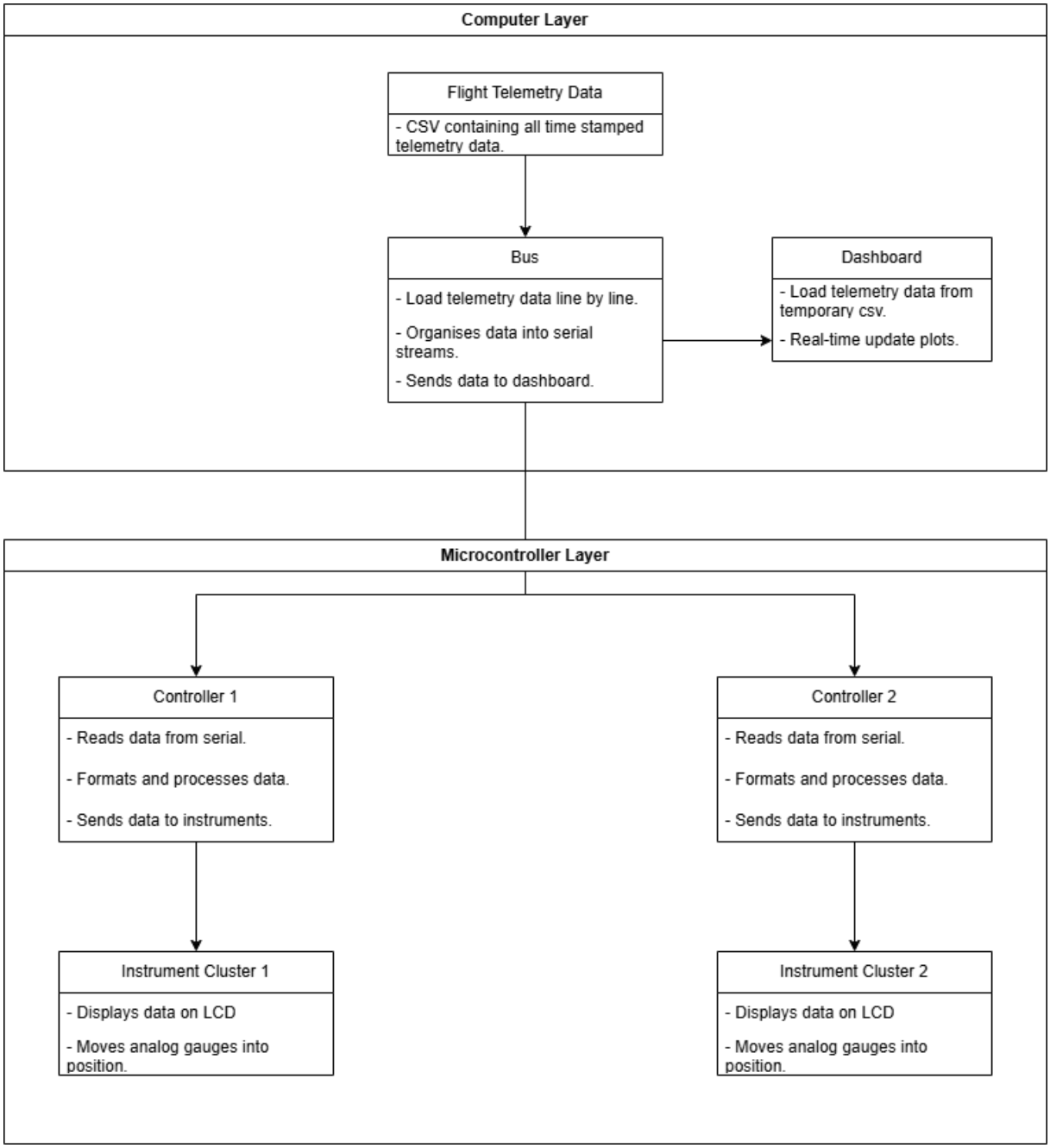


Figure 2: Software Architecture: Data Flow and Component Interaction

Figure 2 illustrates the control flow of the software components. In the Computer Layer, flight telemetry data is stored in the form of a CSV file, which is read and processed by the Bus. The Bus then parses this data into two separate serial streams,

one for each Arduino. It also manages the data stream for the Dashboard by writing the processed flight data to a temporary CSV file that the dashboard can read.

The Microcontroller Layer consists of two Arduinos, each running a program that listens for incoming serial data from the computer. Upon receiving data, the Arduinos update the corresponding gauges and digital displays in real time to reflect the current flight parameters.

This layered architecture enables efficient data handling and a clear separation of responsibilities between the computer and microcontroller components. It also provides flexibility, allowing modules or components to be swapped or extended without requiring a complete rewrite of the codebase. In future iterations, additional functionality could be incorporated, such as a speaker for audio alerts when certain flight limits are reached, or a Bluetooth module to decouple the computer from the microcontrollers and enable a more portable system.

4.3 Software Design Considerations

As mentioned in *The Robotics Primer* by Maja J. Matarić, every Turing-complete programming language can be used to solve any computational problem; however, some languages are better suited for certain tasks than others [5]. For this project, the main software design considerations were safety, performance, scalability, and correctness. For simplicity, I chose to focus on the three languages that offer the strongest support for Arduino development: Python, C/C++, and Rust. The decision matrix in Table 3 compares these languages across the key criteria relevant to this project, using a rating scale of 1 (least suitable) to 3 (most suitable).

Table 3: Decision matrix comparing Python, C/C++, and Rust for Arduinos

Criteria	Python	C/C++	Rust
Ease of Learning	3	2	1
Arduino Support	1	3	2
Performance	1	3	3
Memory Efficiency	1	3	3
Hardware Control	1	3	2
Code Safety / Reliability	2	2	3
Library Availability	2	3	1

After evaluating the three languages based on these criteria, C/C++ emerged as the best overall choice according to the matrix [6]. However, given that Rust ranked a close second and offers significant advantages in terms of memory safety, bug prevention, and modern tooling, as well due to my personal interest in learning it, I chose Rust for the microcontroller programming [7]. For the computer layer, I selected Python because of its ease of use and extensive ecosystem for data processing and visualization. This combination allows me to leverage the strengths of both languages while still meeting the project requirements.

4.4 Testing and Validation

Hardware testing was conducted in stages, starting with individual components and then integrating them into the full system. Each Arduino was tested separately to ensure that the gauges and displays responded correctly to simulated input data. Once both Arduinos were verified, they were connected to the computer, and end-to-end testing was performed to confirm that the entire data flow, from the computer to the microcontrollers, functioned as intended.

Initially the use of formal unit testing frameworks where planned for both the Rust and Python codebases. However, due to time constraints and the complexity of setting up testing for embedded systems, I opted for a more manual testing approach. This involved writing test scripts to simulate flight data and observing the behavior

of the gauges and dashboard in real time. While this method is less rigorous than formal unit testing, it allowed me to validate the core functionality of the system within the project timeline.

Chapter 5

Challenges

5.1 Challenge 1: Lack of Position Feedback

The biggest technical challenge was controlling the DC gear motors without any position feedback. DC gear motors are simple devices that rotate when powered, but they do not provide any information about their current angular position. As a result, when the system powered on, the gauge needles could start from arbitrary positions.

To address this, I attempted to implement proportional control using PWM signals, estimating position indirectly through motor speed and actuation time. However, this approach proved unreliable, as the system frequently overshot or undershot the target positions due to variations in load, friction, and supply voltage.

For future iterations, servo motors would be a more suitable solution for this application, as they provide built-in position feedback and precise control. However, due to the limitations of the development kit and budget constraints, DC motors were used.

5.2 Challenge 2: Crowded Circuit

This project involved a complex circuit consisting of multiple motors, potentiometers, and a display. Initially, I planned to include a physical switch on the board, which would have required using digital pins `D1` to `D13`.

However, during development, I discovered that pins `D0` and `D1` are reserved for USB serial communication. To maintain reliable communication between the Arduino and the computer, these pins needed to remain unused.

As a result, I removed the switch from the design to free up sufficient I/O pins for the remaining components. This trade-off allowed the system to function correctly while staying within the hardware constraints.

5.3 Challenge 3: Logging Data for Real-Time Dashboard

A key goal of this project was to implement a real-time digital dashboard to complement the analog gauges. In the system architecture, the computer generates simulated flight telemetry data and sends it to a central communication bus. This data is then distributed to two Arduino microcontrollers via USB serial.

Initially, I attempted to read the serial data directly from the bus for dashboard logging. However, I found that reading from the serial stream consumed the data, preventing other components from accessing it. Attempts to duplicate or re-read the data resulted in corruption.

To resolve this issue, I implemented a separate data logging pipeline. The telemetry data is written to a CSV file, which is then read exclusively by the dashboard. This approach ensured data integrity while allowing both the embedded system and the dashboard to operate reliably without interfering with each other.

Chapter 6

Results and Discussion

In practice, communication between the computer and the Arduinos proved to be reliable, with no significant instances of data loss or corruption observed during testing. Basic error handling was implemented on the Arduino side to account for potential parsing issues, such as non-numeric input or incomplete data packets. This included validating incoming float values and ensuring that the expected number of data fields were received before updating the gauges. Overall, the communication system was robust and sufficient for the requirements of this project.

On the hardware side, the circuit was relatively robust and performed reliably throughout testing. Gauge movements were smooth; however, they were primarily qualitative, as the readings were not fully accurate due to the absence of position feedback from the DC motors. In contrast, the digital displays provided accurate representations of the flight parameters, and the system responded consistently to changes in the input data. The overall setup is shown in Figure 3, which illustrates the physical arrangement of components and wiring from both top and front views.

(a) Front view of the flight emulator controller.

(b) Top view of the flight emulator controller.

Figure 3: Flight emulator controller: (a) front view and (b) top view.

One limitation of the current system is that it is not yet optimized for deployment or distribution. Setup requires several manual steps, including configuring USB serial ports and ensuring synchronized communication between the Arduinos and the Bus through terminal commands. Future improvements could focus on automating this initialization process to improve usability and portability.

The dashboard visualization was tested, and the plotting and live updating matched the digital display values. The `steamlit` library in Python proved to be very convenient for creating a simple dashboard interface, and the live updating of the plots [8].

Chapter 7

Conclusion

7.1 Lessons Learned

This project provided valuable insight into the challenges of integrating hardware and software systems in a real-time environment. One of the key lessons learned was the importance of working within hardware constraints. The lack of position feedback from the DC motors required alternative solutions, such as the implementation of a manual homing.

Another important takeaway was the role of system architecture in managing complexity. Separating the system into a Computer Layer and a Microcontroller Layer allowed for clearer responsibilities, improved scalability, and easier debugging. This modular approach proved essential when integrating multiple subsystems, including serial communication, motor control, and data visualization.

7.2 Future Improvements

Several areas for improvement have been identified for future iterations of this system. The most significant enhancement would be the integration of position feedback, either through the use of servo motors or the addition of encoders to the

existing DC motors. This would enable true closed-loop control and significantly improve the accuracy of the analog gauges.

From a system design perspective, the current setup could be optimized for usability and portability. Automating the initialization process, including serial port configuration and synchronization between components, would reduce setup complexity and make the system more user-friendly. The addition of wireless communication, such as Bluetooth, could further decouple the system from the computer and improve portability.

On the software side, implementing formal testing frameworks would improve reliability and maintainability. Expanding the dashboard functionality, including additional visualization features and user interaction, would also enhance the overall user experience. Finally, refining the mechanical design of the gauge system to include an actual scale from $0° - 360°$ and incorporating it into a 3D printed enclosure would refine the project further.

7.3 Final Thoughts

Overall, this project successfully demonstrates the feasibility of a hardware-software integrated flight instrumentation system. The combination of motorized analog gauges, digital displays, and a real-time dashboard creates a tangible and interactive representation of flight data. While the system has limitations, particularly regarding accuracy and software deployability, it achieves its core objectives.

Beyond the technical outcomes, this project provided valuable experience in embedded systems development within an aerospace context, allowing me to join mechanical engineering and computer science into a working system.

APPENDIX A: Media and Resources

This section provides supplementary media and resources related to the project, including the source code repository and demonstration videos.

GitHub Repository:

The full codebase and supporting documentation for this project are available at the following repository:

https://github.com/kenyikubari/flight-emulator-project

Video Demonstrations:

The following videos showcase different aspects of the flight emulator system:

- **Instrument Cluster Demonstration:**
  https://youtu.be/lIG7tXPKBvI?si=PpsRivrkwLYHKv8j

- **Serial Data Stream Demonstration:**
  https://youtu.be/hduWfnMPjoY

- **Dashboard Visualization Demonstration:**
  https://youtu.be/hO7zURT9W1o

APPENDIX B: Documentation and Schematics

This Section contains the documentation and schematics for the project, including the Arduino RedBoard V2.2 schematic, TB6612FNG motor driver specification sheet, and the flight emulator circuit schematics. These resources provide a overview of the hardware components.

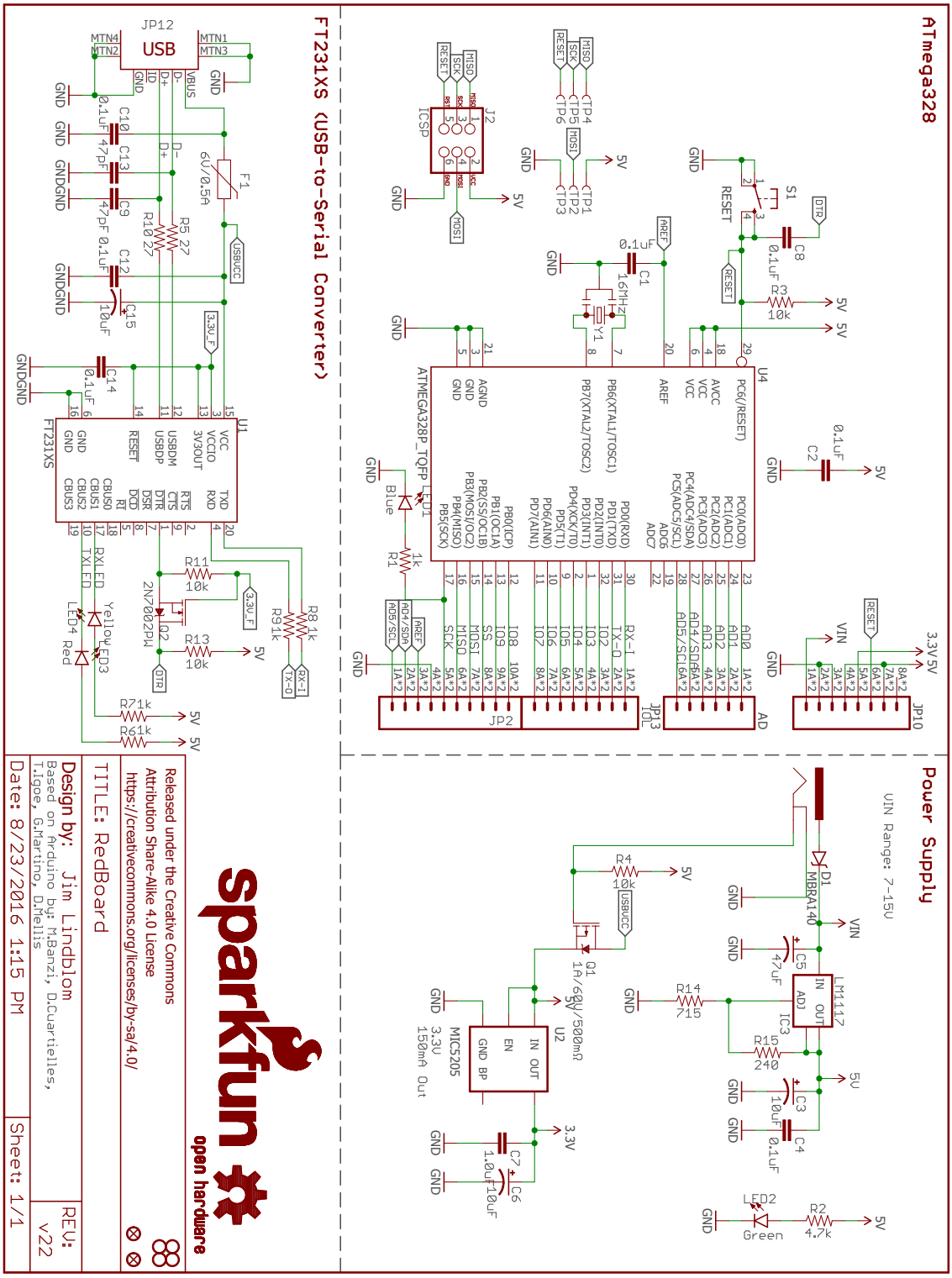


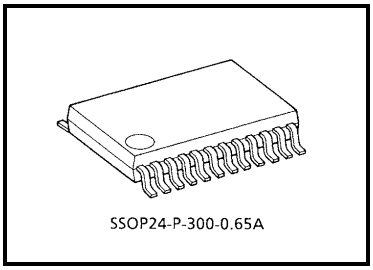
Figure 4: Arduino RedBoard V2.2 schematic

Toshiba Bi-CD Integrated Circuit Silicon Monolithic

T B 6 6 1 2 F N G

Driver IC for Dual DC motor

TB6612FNG is a driver IC for DC motor with output transistor in LD MOS structure with low ON-resistor. Two input signals, IN1 and IN2, can choose one of four modes such as CW, CCW, short brake, and stop mode.



質量: 0.14 g (標準)

Features

- Power supply voltage ； VM=15V（Max.）
- Output current ； Iout=1.2A(ave) / 3.2A (peak)
- Output low ON resistor； 0.5Ω (upper＋lower Typ. @VM≧5V)
- Standby (Power save) system
- CW/CCW/short brake/stop function modes
- Built-in thermal shutdown circuit and low voltage detecting circuit
- Small faced package（SSOP24：0.65mm Lead pitch）
- Response to Pb free packaging

* This product has a MOS structure and is sensitive to electrostatic discharge. When handling this product, ensure that the environment is protected against electrostatic discharge by using an earth strap, a conductive mat and an ionizer. Ensure also that the ambient temperature and relative humidity are maintained at reasonable levels.

The TB6612FNG is a Pb-free product.
The following conditions apply to solderability:
*Solderability
1. Use of Sn-37Pb solder bath
   *solder bath temperature = 230°C
   *dipping time = 5 seconds
   *number of times = once
   *use of R-type flux
2. Use of Sn-3.0Ag-0.5Cu solder bath
   *solder bath temperature = 245°C
   *dipping time = 5 seconds

Figure 5: TB6612FNG Motor Driver Spec Sheet [1].

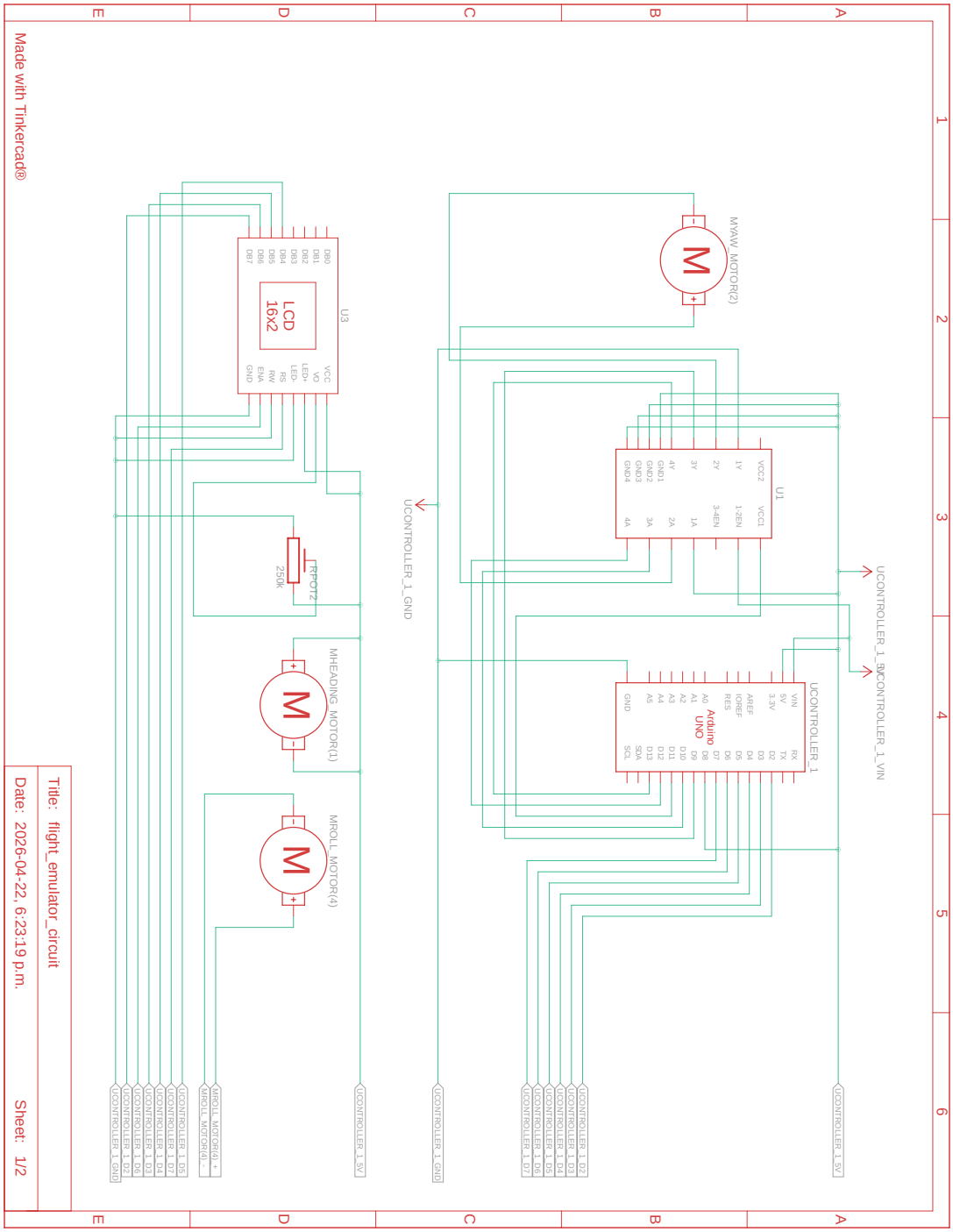


Figure 6: Flight Emulator Circuit Schematic Page 1

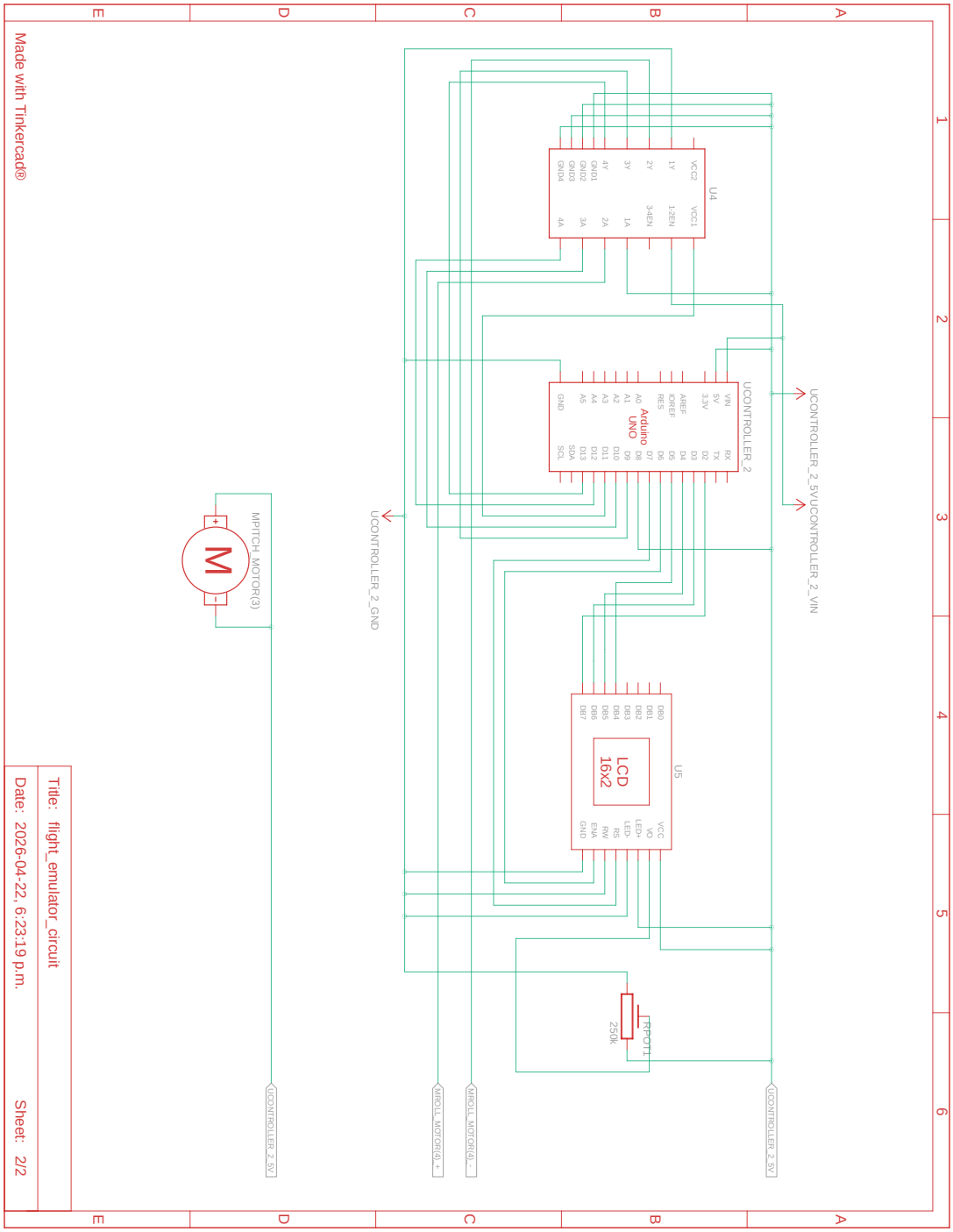


Figure 7: Flight Emulator Circuit Schematic Page 2

APPENDIX B: Bibliography

[1] Toshiba, "TB6612FNG Dual DC Motor Driver IC Datasheet," Datasheet, June 2007. [Online]. Available: https://cdn.sparkfun.com/datasheets/Robotics/TB6612FNG.pdf

[2] Transport Canada, "Flight Test Guide – Instrument Rating Groups 1, 2 and 3 – Aeroplane (TP 9939E)." [Online]. Available: https://tc.canada.ca/en/aviation/publications/flight-test-guide-instrument-rating-groups-1-2-3-aeroplane-tp-9939e

[3] University of Manitoba, Department of Mechanical Engineering, "MECH 4182 Aerospace Structures: Analysis and Design." 2026.

[4] University of Manitoba, Department of Mechanical Engineering, "MECH 3520 Aerodynamics." 2026.

[5] M. J. Matarić, *The Robotics Primer*, 2nd ed. Cambridge, MA: MIT Press, 2017.

[6] Arduino, "Arduino Documentation." 2026.

[7] Naxaes, "rust-arduino: Playing with Rust on Arduino." 2026.

[8] Streamlit Inc., "Streamlit Documentation." 2026.